

ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES DE TÉTOUAN

Filière : Cycle d'Ingénieur en Génie Informatique S6

Application de Messagerie Instantanée avec appels audio et vidéo

Réalisé par :

Abdellah **BOUARGUAN**

Nizar **BEN YACoub**

Marouane **EL GHAZOUANI**

Nafissa **EL YOUNOUSSI**

Leila **BOUALAM**

Sous la direction de :

Pr. EL BOUHDIDI Jaber

30 avril 2026

Table des matières

1	Introduction	3
2	Analyse et Conception (UML)	3
2.1	Cas d'Utilisation	3
2.2	Diagramme de Classes	5
2.3	Diagramme de Séquence	6
2.4	Diagramme de Déploiement	7
3	Analyse des Modules (Architecture Technique)	9
3.1	Module Commun (<code>com.messenger.common</code>)	9
3.2	Module Client (<code>com.messenger.client</code>)	9
3.3	Module Interface Utilisateur Client (<code>com.messenger.client.ui</code>)	9
3.4	Module Média Client (<code>com.messenger.client.media</code>)	9
3.5	Module Serveur (<code>com.messenger.server</code>)	9
4	Flux Système	9
4.1	Flux d'Authentification et d'Inscription	9
4.2	Flux de Messagerie	9
4.3	Flux de Signalisation et de Streaming Média P2P	10
5	Conclusion	10

1 Introduction

Ce document présente l'architecture logicielle et les choix de conception d'une application de messagerie instantanée, conçue pour simuler les fonctionnalités de base de WhatsApp. L'objectif principal de ce projet est de mettre en œuvre les concepts fondamentaux de la programmation orientée objet et de la conception UML, au travers d'une architecture client-serveur robuste et multi-threadée. L'application repose sur l'utilisation avancée des threads et des sockets réseaux (TCP et UDP), permettant une communication fluide en temps réel. L'interface utilisateur est développée à l'aide du framework JavaFX, offrant une expérience moderne et réactive. L'architecture est clairement séparée entre les composants client et serveur. Les fonctionnalités incluent l'authentification, le suivi de présence en temps réel, l'échange de messages textuels, et les communications multimédias (appels audio/vidéo) en Peer-to-Peer. Le serveur agit comme un routeur central utilisant des sockets TCP standards, tandis que les données de l'application sont persistées au sein d'une base de données PostgreSQL via JDBC.

2 Analyse et Conception (UML)

La conception du système a été modélisée en respectant rigoureusement les standards UML. Pour assurer une lisibilité optimale, les diagrammes complexes sont présentés sur des pages dédiées en pleine page.

2.1 Cas d'Utilisation

Le système interagit principalement avec l'Utilisateur final et un acteur système secondaire, la Base de données.

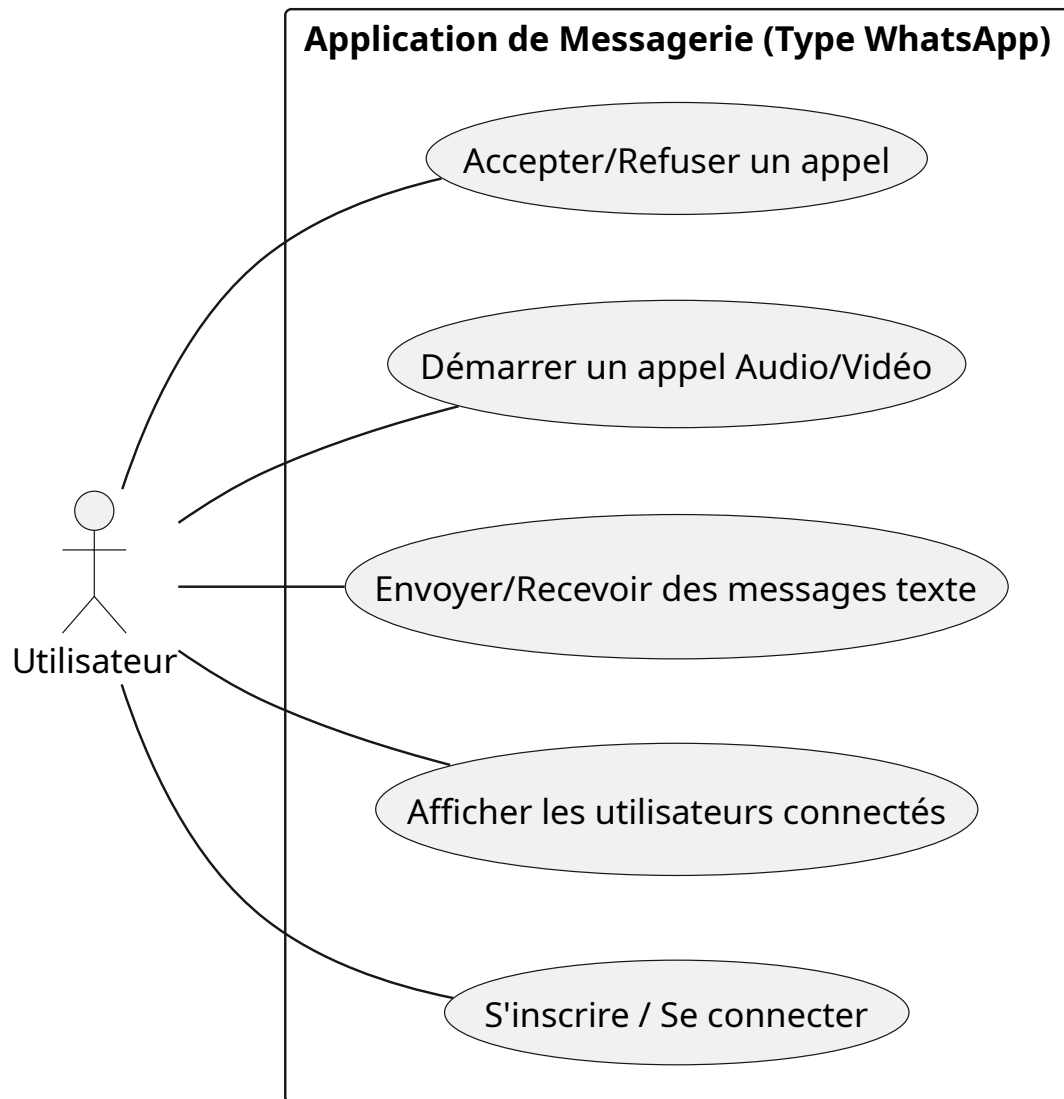
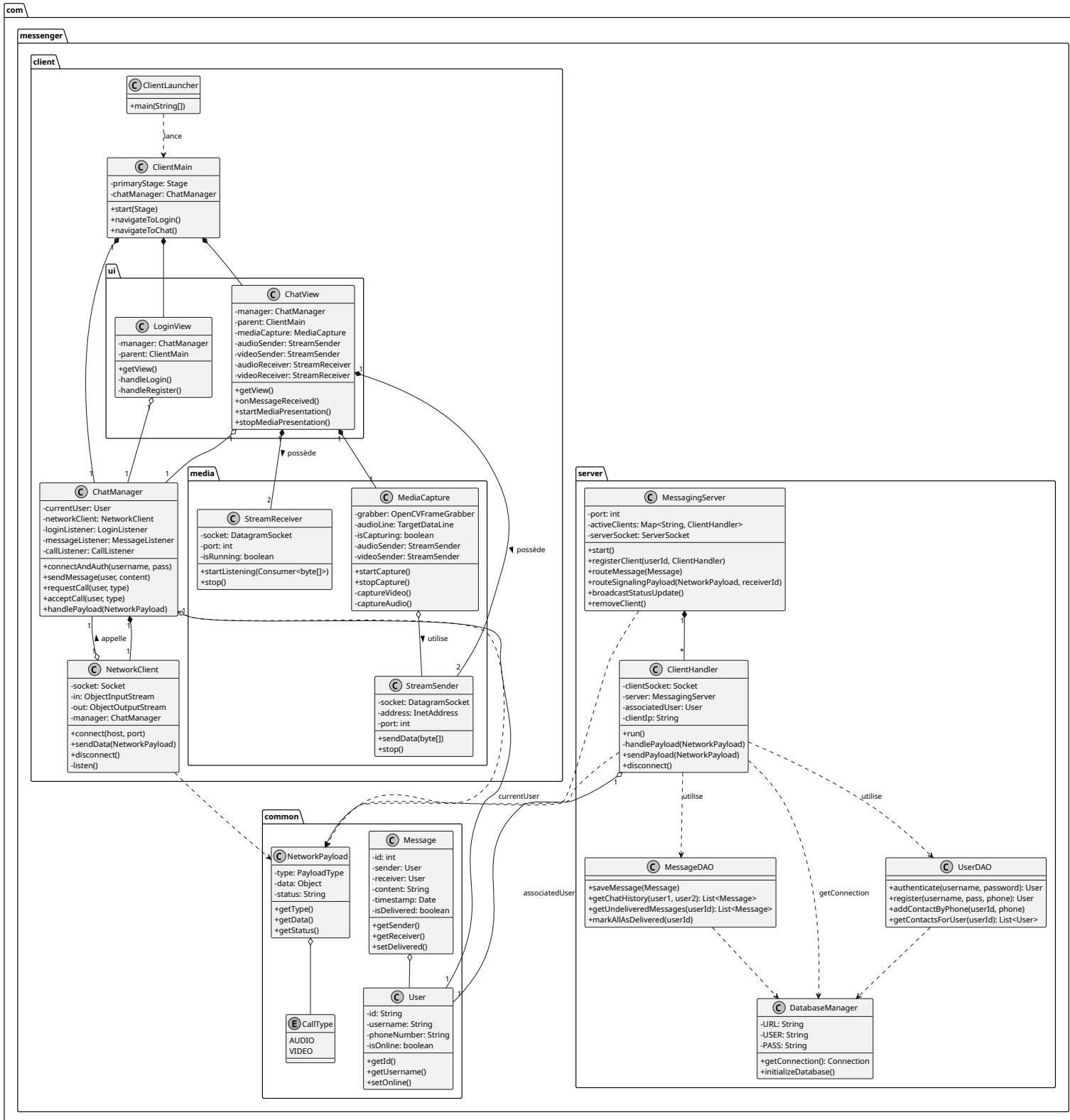
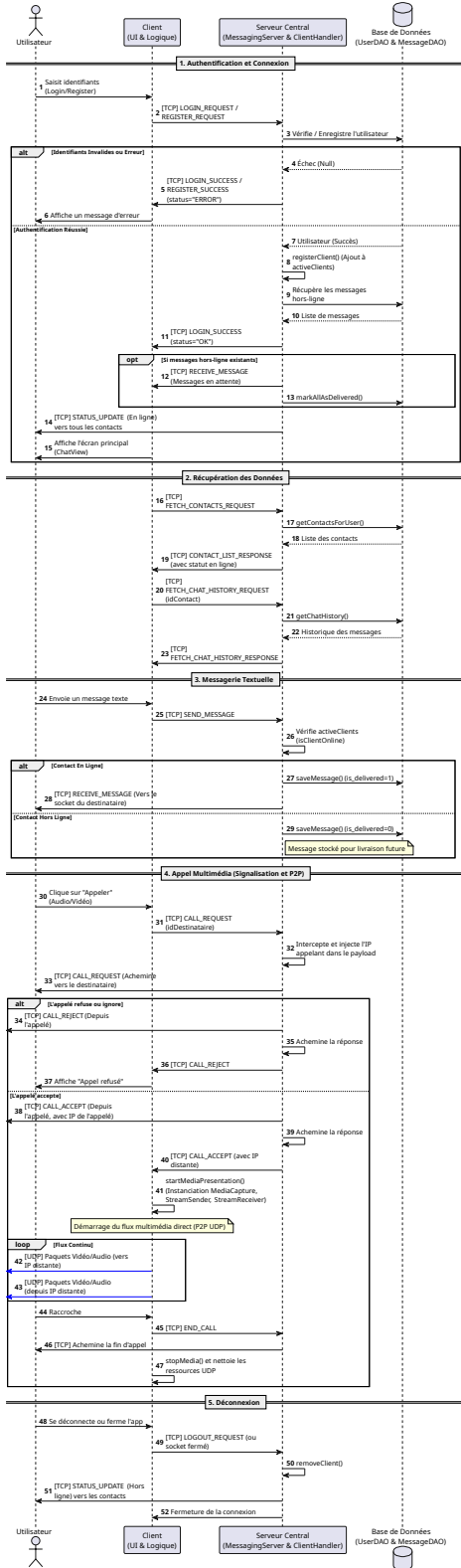


FIGURE 1 – Diagramme des Cas d'Utilisation du système

2.2 Diagramme de Classes

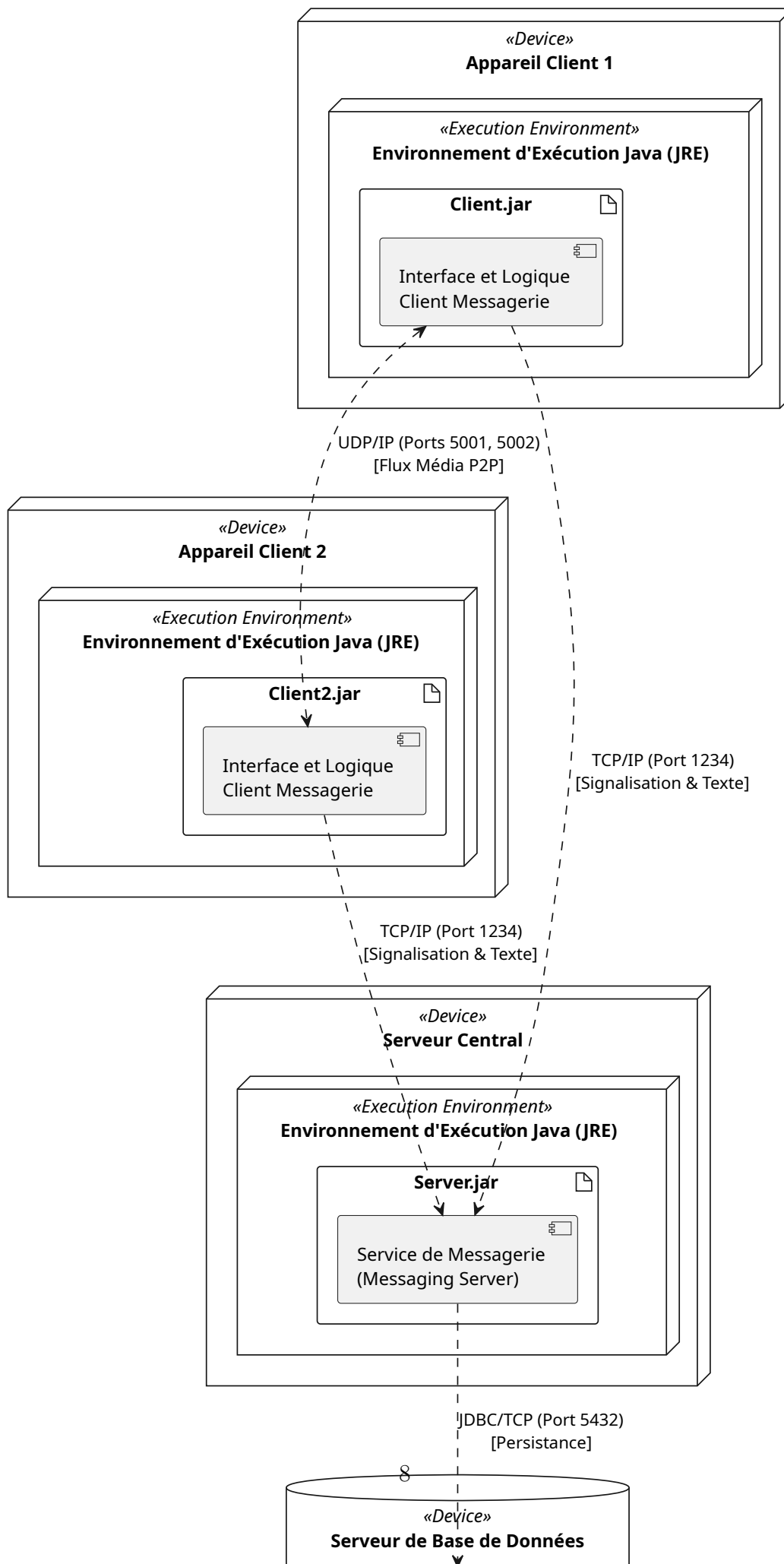


2.3 Diagramme de Séquence



2.4 Diagramme de Déploiement

Ce diagramme illustre la topologie physique du réseau et les protocoles de communication.



3 Analyse des Modules (Architecture Technique)

3.1 Module Commun (`com.messenger.common`)

Le package `common` agit comme la couche de données fondamentale partagée entre le client et le serveur.

- `User.java` & `Message.java` : Ce sont des POJO standards implémentant l'interface `Serializable`.
- `CallType.java` : Une énumération définissant les types d'appels `AUDIO` et `VIDEO`.
- `NetworkPayload.java` : Cette classe implémente le patron de conception "Wrapper".

3.2 Module Client (`com.messenger.client`)

Ce module constitue l'épine dorsale de la logique de l'application cliente.

- `NetworkClient.java` : Gère la connexion socket TCP avec le serveur.
- `ChatManager.java` : L'orchestrateur principal côté client.

3.3 Module Interface Utilisateur Client (`com.messenger.client.ui`)

- `LoginView.java` : Authentification et inscription.
- `ChatView.java` : Interface de messagerie et contrôles multimédias.

3.4 Module Média Client (`com.messenger.client.media`)

- `StreamSender.java` / `StreamReceiver.java` : Flux UDP.
- `MediaCapture.java` : Capture (JavaSound, OpenCV).

3.5 Module Serveur (`com.messenger.server`)

Le composant centralisé de routage et de gestion d'état.

- `MessagingServer.java` : Démon principal.
- `ClientHandler.java` : Thread dédié à chaque client.
- `DatabaseManager.java` : Connexion JDBC PostgreSQL.

4 Flux Système

4.1 Flux d'Authentification et d'Inscription

L'utilisateur s'authentifie via `LoginView`. Le serveur valide via `UserDAO` et diffuse le statut "En ligne".

4.2 Flux de Messagerie

Le serveur route les messages en temps réel (TCP) ou les stocke en base de données si le destinataire est hors-ligne.

4.3 Flux de Signalisation et de Streaming Média P2P

La signalisation passe par le serveur pour l'échange d'adresses IP. Le flux multimédia (UDP) s'établit ensuite directement entre les clients pour une performance maximale.

5 Conclusion

La version 1 de cette application démontre une architecture hybride efficace. La séparation des préoccupations et le choix des protocoles (TCP/UDP) assurent fiabilité et réactivité.